

**7. Header Linked List**  
**–Delete From Beginning**

**Program:**

```
void deleteFromBeginning()
{
    if (head->next == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head->next; // The first actual node

    // Update header to point to the second node
    head->next = temp->next;

    cout << "Deleted " << temp->data << " from the
beginning." << endl;

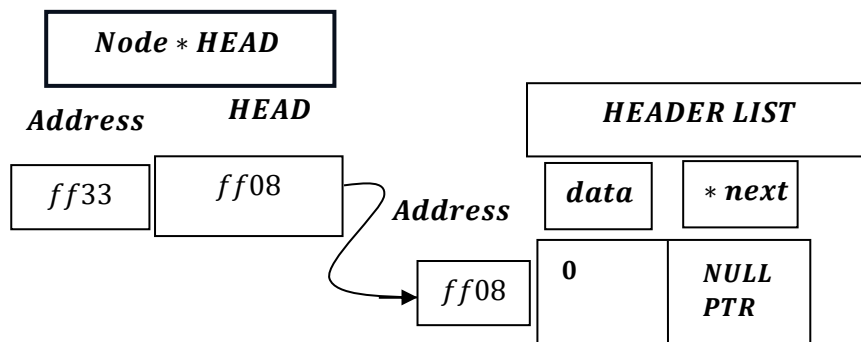
    free(temp);
    temp = nullptr;

    // Decrement internal count
    head->data--;
}
...
    case 5:
        deleteFromBeginning();
        break;
```

## Program Analysis

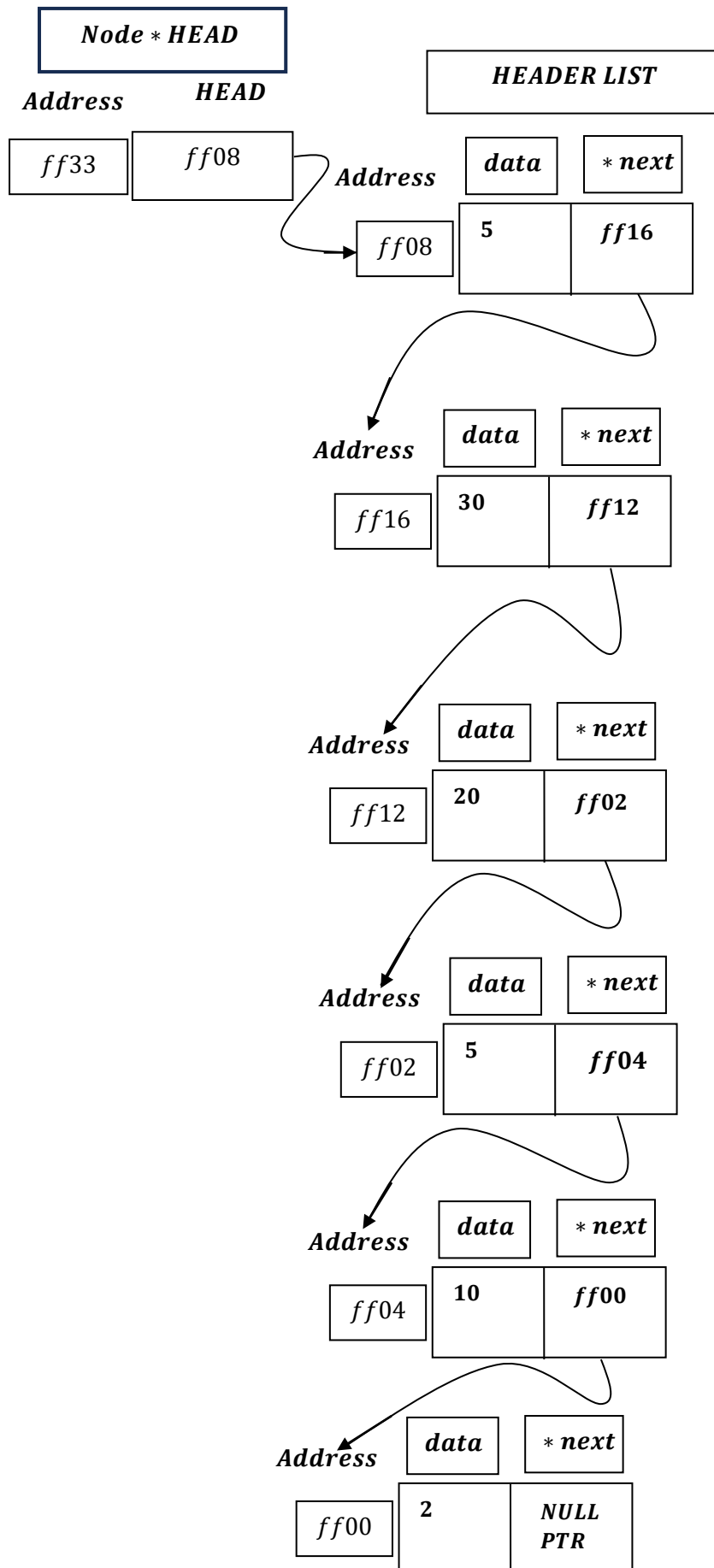
```
if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

*i.e.*



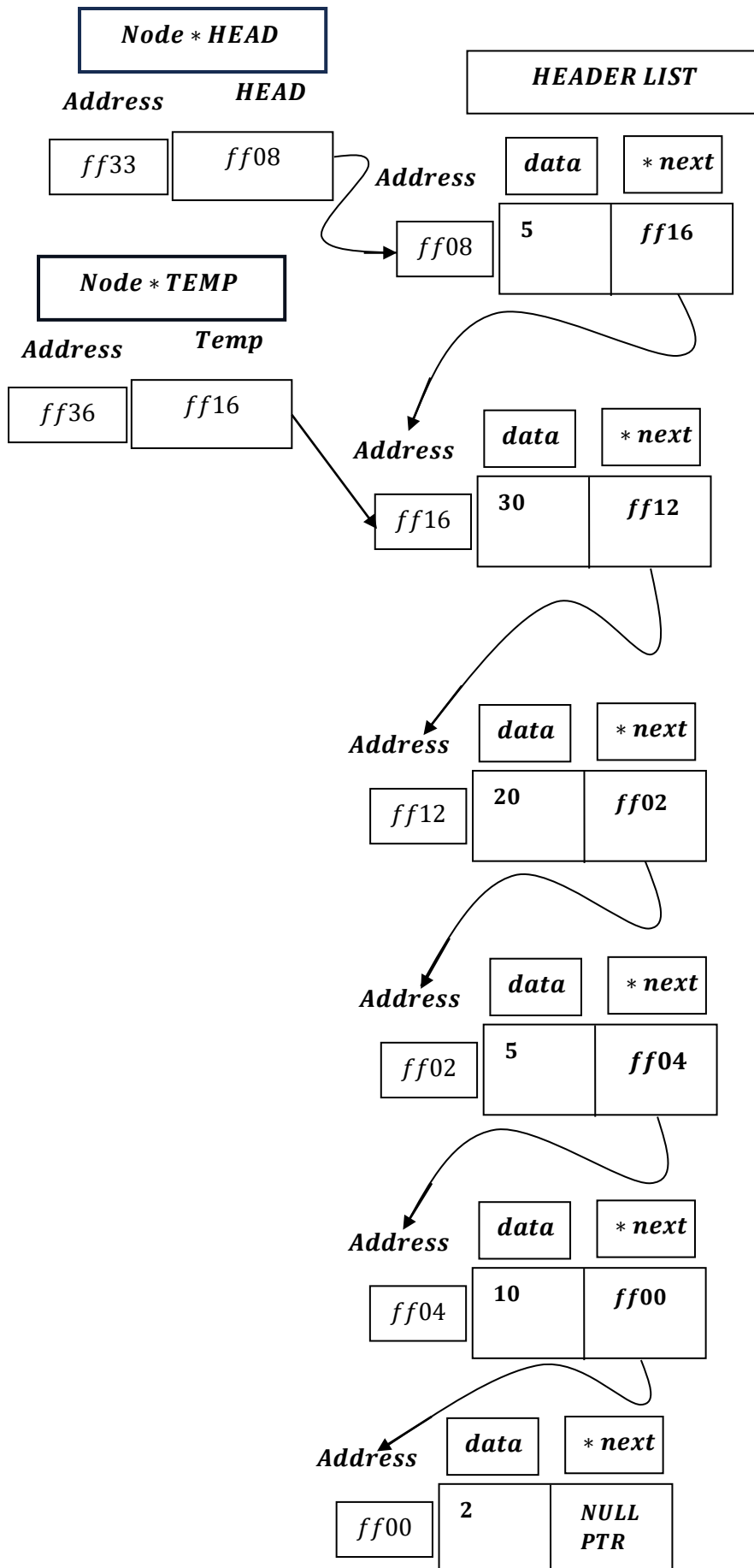
*head* → *next* = *NULLPTR* , *head* → *Data* = 0 , then list is empty .

## *If List is not Empty*



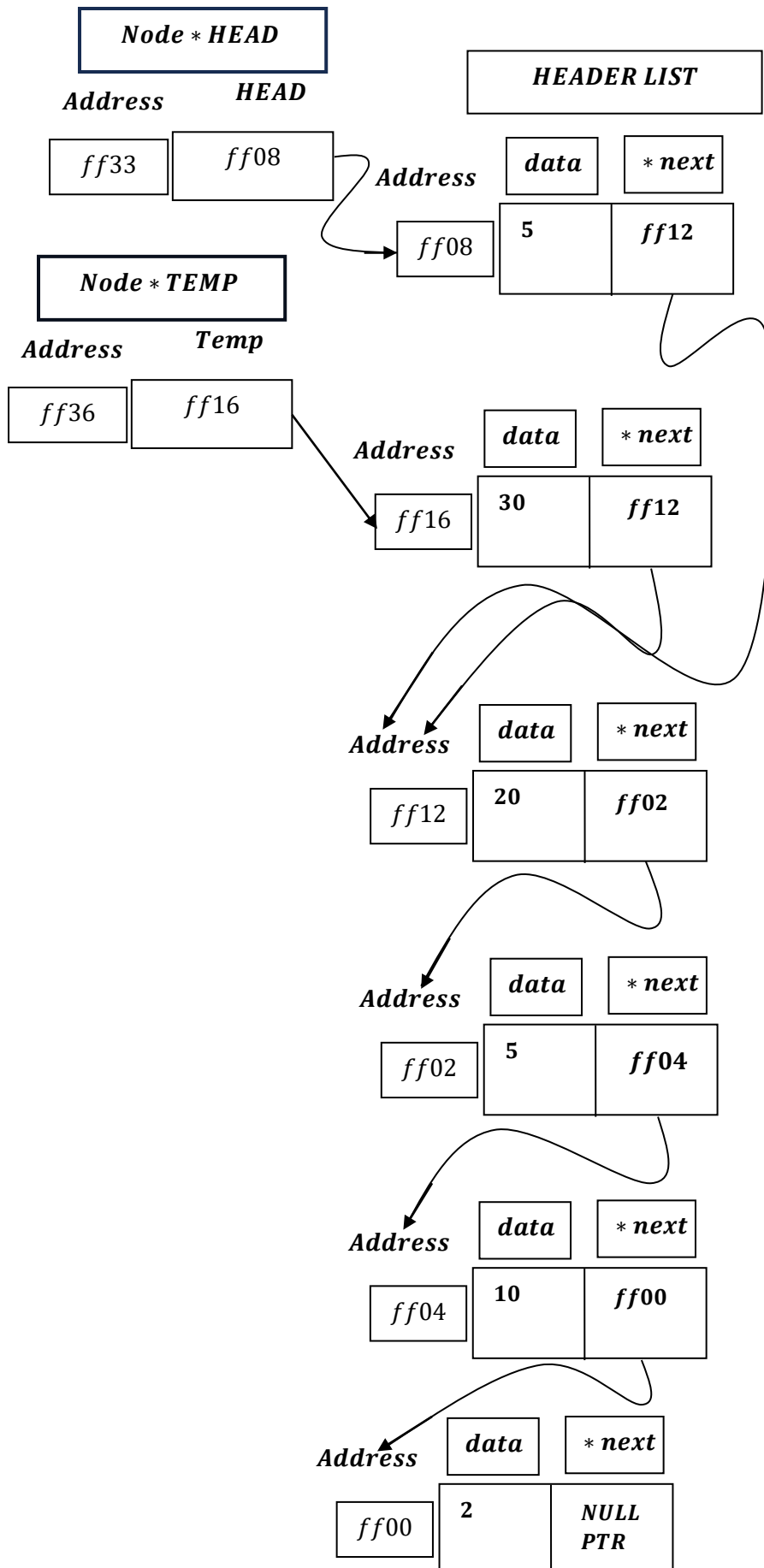
```
Node *temp = head->next; // The first actual node
```

*Temp = Head → Next = ff16.*



```
head->next = temp->next;
```

$Head \rightarrow Next = Temp \rightarrow Next = ff12.$

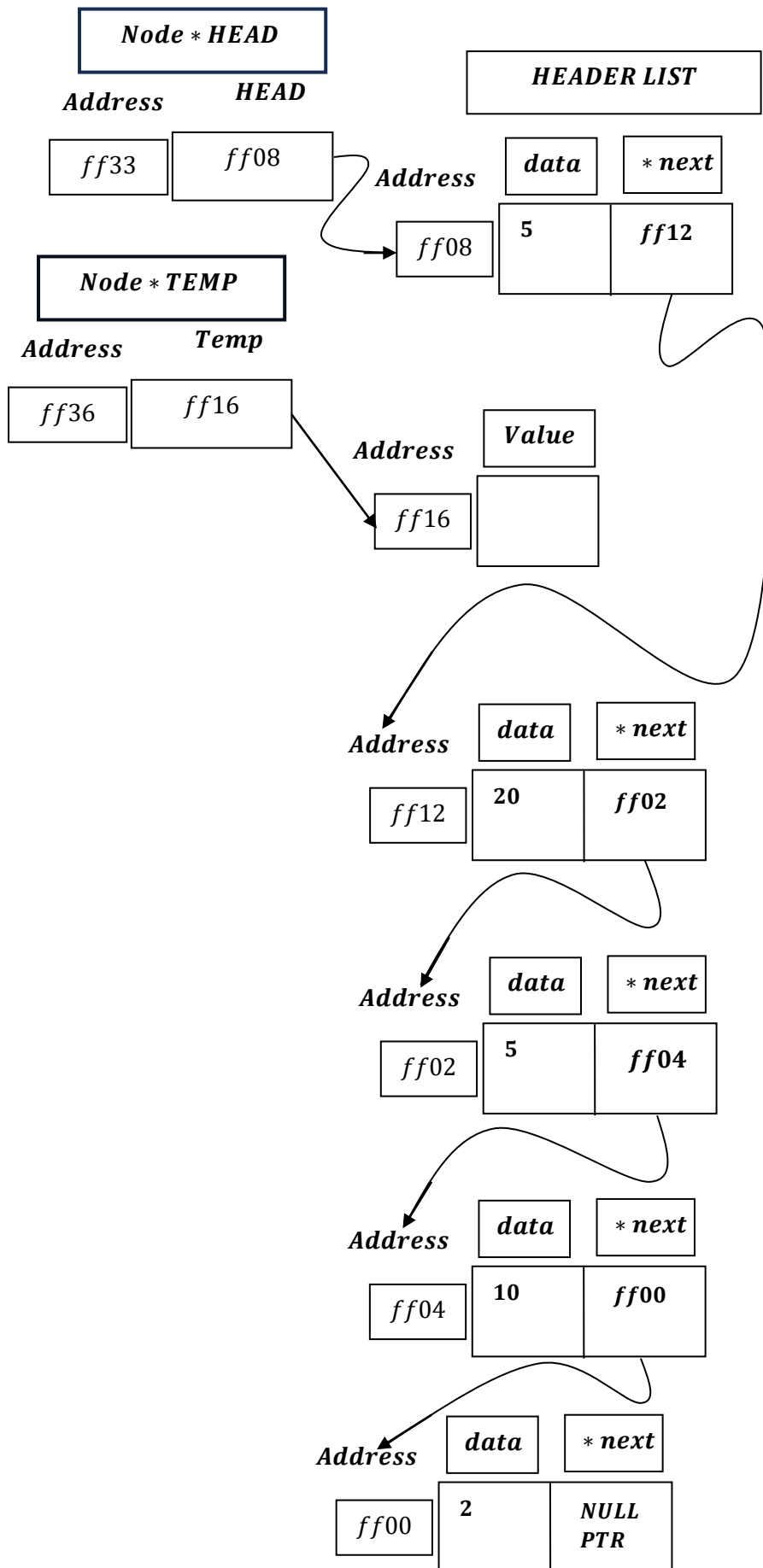


```
cout << "Deleted " << temp->data << " from the  
beginning." << endl;
```

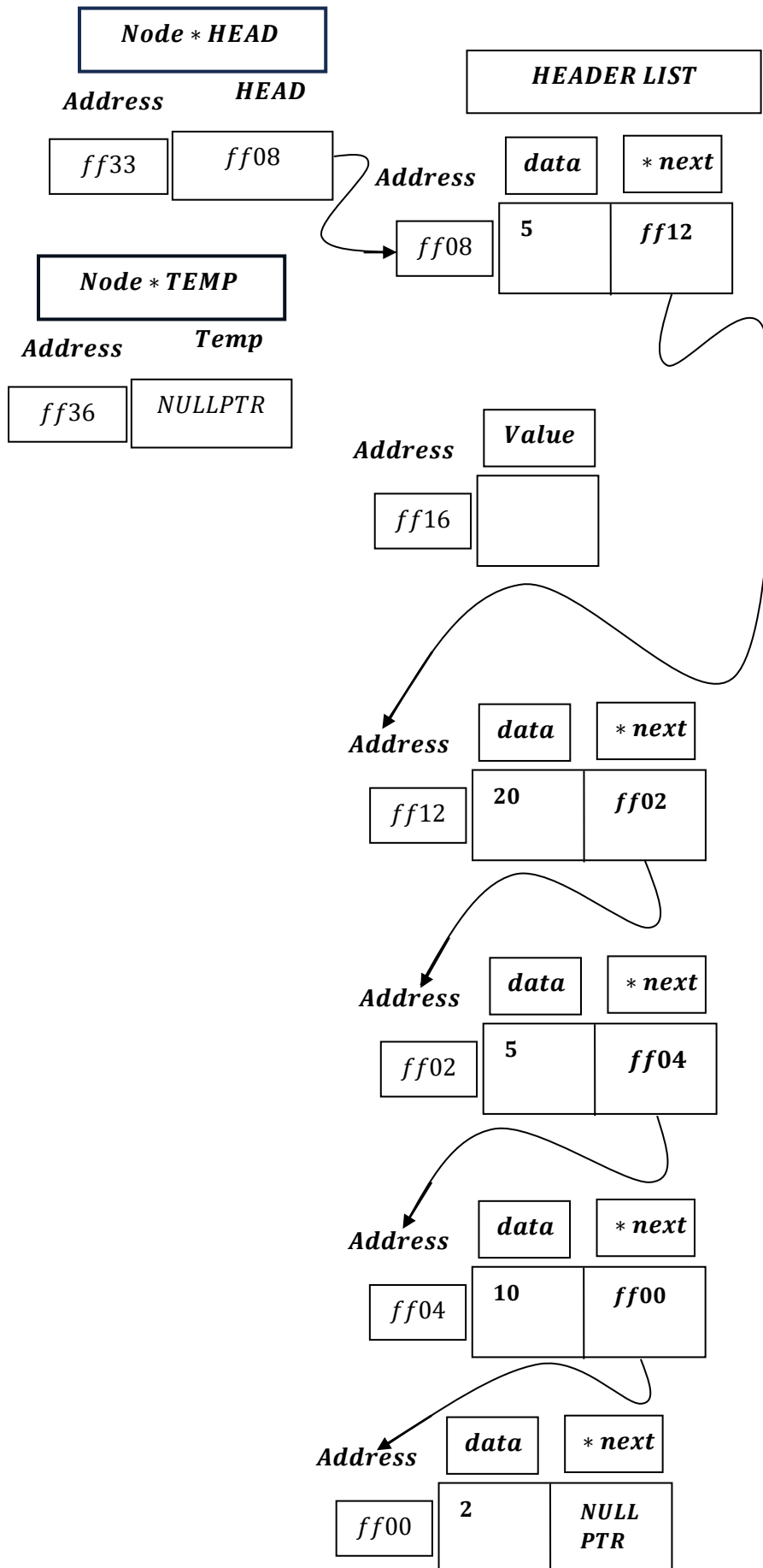
*Prints: Deleted ``Temp → Data: 30`` from the beginning.*

```
free(temp);
```

*It will free the memory that TEMP pointer that it point to .*



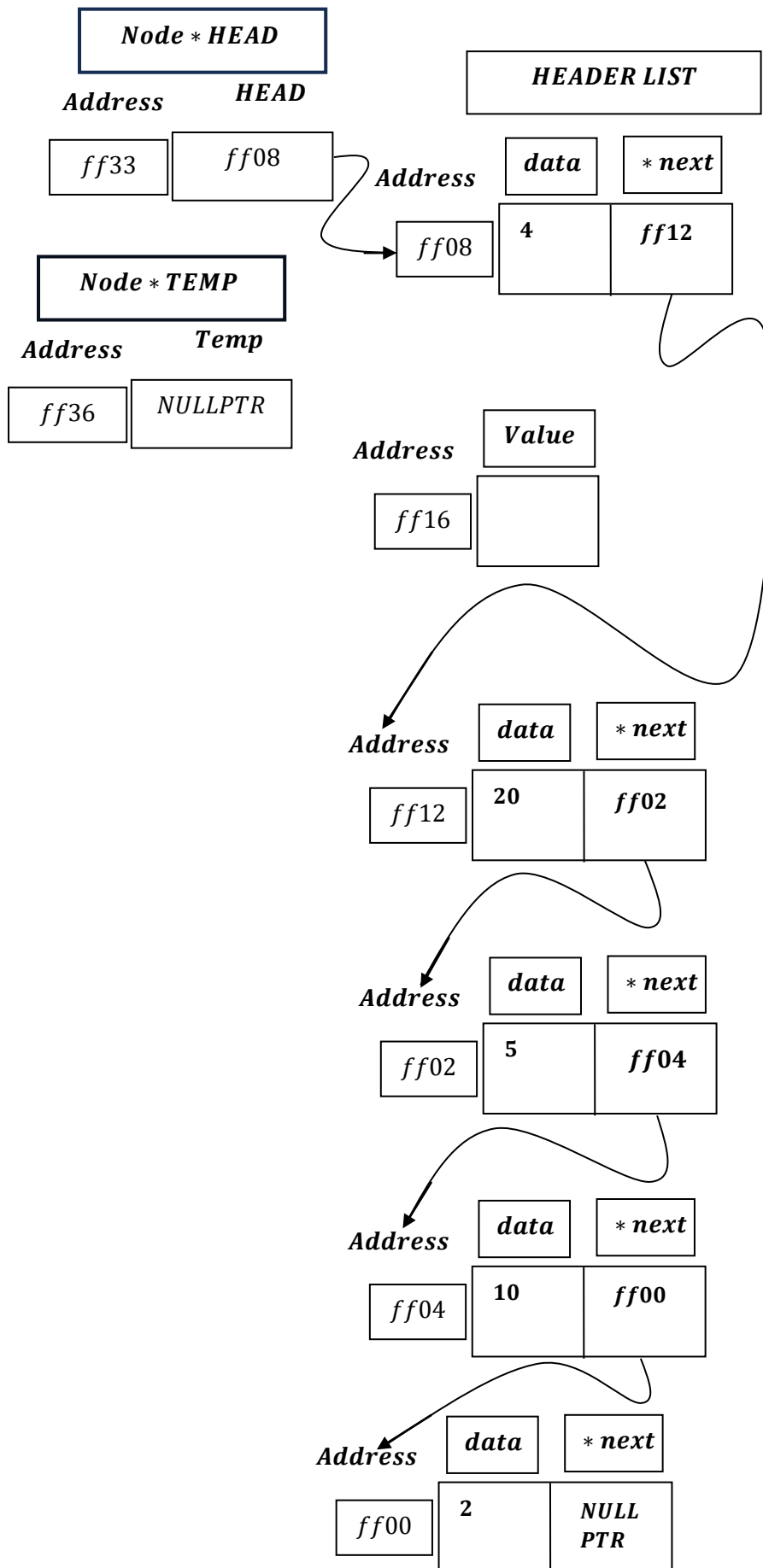
```
temp = nullptr;
```





`head->data--;`

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 5 - 1 = 4.$



*And we get our list:*

